PYTHON SUMMER PLAN • DAY 01

OOP Fundamentals

Classes, __init__, attributes, methods, __repr__ / __str__

Monday, May 11 · 1 hour

Yuvraj Ruparel

What you'll learn today

Classes

Your own data types — bundle state and behavior together.

`__init__`

The constructor — runs automatically when you create an instance.

Instance vs class attrs

Per-object data vs shared-across-all-instances data.

Methods

Functions defined inside a class — they know about self.

`__repr__` and `__str__`

Make print(obj) produce something useful, not a memory address.

Functions vs Objects: why bother?

FUNCTION APPROACH

```
# Pass 5 args every time
def simulate(m, k, c, x0, v0, t):
    # ...
    return x, v

x, v = simulate(1, 10, 0.3, 1, 0, 10)
```

State lives loose. You drag every parameter through every call. Easy to mix up.

OBJECT APPROACH

```
smd = SpringMassDamper(
    m=1, k=10, c=0.3,
    x0=1, v0=0)

x, v = smd.simulate(10)
smd.plot()
```

State is bundled into one object. You build it once, then ask it to do things.

Takeaway

An object is a piece of state that knows how to behave. OOP lets your code mirror the way you already think about physical systems: a spring-mass-damper, a pendulum, a sensor.

The simplest class

```
class Dog:
    pass

d = Dog()
print(d)
```

OUTPUT

```
<__main__.Dog object at 0x7f8a1c003d50>
```

What just happened?

`class Dog:`
defines a new type called Dog.

`Dog()`
creates an instance — a specific dog.

But it has no data and no behavior yet.

The ugly output is the point

That memory address is Python telling you: "I made an object, but I have no idea how to describe it." We'll fix that with __init__ and __repr__.

__init__ — bringing the object to life

```python
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

d = Dog("Rex", 5)
print(d.name, d.age)
```

OUTPUT

```
Rex 5
```

Three things to internalize

`__init__`

runs automatically when you write Dog(...).

`self`

is the specific instance being built right now.

`self.x = x`

stores x as an attribute on that instance.

Why "self"? Python doesn't pass it magically — when you call `d.name`, Python translates it to `Dog.name(d)` behind the scenes.

Instance vs class attributes

```python
class Dog:
    species = "Canis familiaris"   # class attribute

    def __init__(self, name):
        self.name = name           # instance attribute

a = Dog("Rex")
b = Dog("Buddy")
print(a.name, b.name)
print(a.species, b.species)
```

OUTPUT

```
Rex Buddy
Canis familiaris Canis familiaris
```

When to use which

INSTANCE

Unique per object. Set in __init__ via self.x = ...

Example: mass, position, velocity of a specific particle.

CLASS

Shared by all instances. Defined directly in the class body.

Example: g = 9.81, π, molar masses, default colors.

⚠️ *Mutating a class attribute (e.g. a shared list) affects every instance. Stick to immutable defaults.*

Methods — functions that know about self

```python
import math

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

    def circumference(self):
        return 2 * math.pi * self.radius
```

```python
c = Circle(5)
print(c.area())
```

OUTPUT

```
78.53981633974483
```

Method anatomy

- Defined inside the class, indented.
- First parameter is always self.
- Access instance data via self.attr.
- Called with object.method() — Python passes self automatically.

__repr__ — kill the memory address

BEFORE

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

OUTPUT

```
>>> print(Dog("Rex", 5))
<__main__.Dog object at 0x7f8a...>
```

AFTER

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"Dog({self.name!r}, {self.age})"
```

OUTPUT

```
>>> print(Dog("Rex", 5))
Dog('Rex', 5)
```

Rule of thumb

__repr__ should be unambiguous — ideally you could paste the output back into Python and recreate the object. The !r in f-strings adds quotes around strings automatically.

__str__ vs __repr__

```python
class Dog:
    def __init__(self, name, age):
        self.name, self.age = name, age

    def __repr__(self):
        return f"Dog({self.name!r}, {self.age})"

    def __str__(self):
        return f"{self.name}, age {self.age}"
```

OUTPUT

```
>>> d = Dog("Rex", 5)
>>> print(d)        # uses __str__
Rex, age 5
>>> repr(d)         # uses __repr__
"Dog('Rex', 5)"
```

Audience matters

`__str__`

For users. Used by print() and str(). Should be readable.

`__repr__`

For developers. Used by the REPL and repr(). Should be unambiguous.

If you only define __repr__, Python uses it for both. Always start there.

All five concepts in one class

```python
import math

class Particle:
    dim = 2                              # class attribute

    def __init__(self, x, y, mass):
        self.x = x                       # instance attributes
        self.y = y
        self.mass = mass

    def distance_from_origin(self):      # method
        return math.sqrt(self.x**2 + self.y**2)

    def __repr__(self):                  # dunder
        return f"Particle(x={self.x}, y={self.y}, m={self.mass})"
```

```python
p = Particle(3, 4, 1.5)
print(p)
print(p.distance_from_origin())
print(p.dim)
```

OUTPUT

```
Particle(x=3,
  y=4, m=1.5)

5.0

2
```

You can now build classes that model physical systems. Time to write one.

TODAY'S EXERCISE · 45 MIN

Build a Vector3D class

A reusable 3D vector primitive. You'll need this in every physics project you write this summer — and it's the perfect first object because the math is unambiguous and you already know it.

KEY FORMULAS

Magnitude

$$\|v\| = \sqrt{(x^2 + y^2 + z^2)}$$

Dot product

$$v \cdot u = x_1x_2 + y_1y_2 + z_1z_2$$

Normalize

$$\hat{v} = v / \|v\|$$

Returns a new vector — does not mutate the original.

Requirements

1. `__init__(self, x, y, z)`

   Store x, y, z as instance attributes. Accept ints or floats.

2. `magnitude(self)`

   Return $\sqrt{(x^2 + y^2 + z^2)}$ as a float.

3. `normalize(self)`

   Return a NEW Vector3D with the same direction and magnitude 1.

4. `dot(self, other)`

   Return the dot product of self and another Vector3D.

5. `__repr__(self)`

   Return a string like Vector3D(1.0, 2.0, 3.0).

*Use math.sqrt or x**0.5 — either is fine. Don't reach for NumPy yet; the point is to write the math yourself.*

Done when this script works

```python
from vector3d import Vector3D

v = Vector3D(1, 2, 3)
print(v)
print(v.magnitude())

u = Vector3D(1, 0, 0)
w = Vector3D(0, 1, 0)
print(u.dot(w))

n = v.normalize()
print(n)
print(n.magnitude())
```

OUTPUT

```
Vector3D(1, 2, 3)
3.7416573867739413

0

Vector3D(0.267..., 0.534..., 0.801...)
0.9999999999999999
```

Floating-point note

A normalized vector's magnitude often prints as 0.9999...something. That's normal — floating-point precision, not a bug.

Tips & gotchas

Don't forget self.

Every method's first parameter is self. Every attribute access inside the class is self.attr — not just attr.

normalize() returns a NEW vector.

Don't mutate self. Create and return a fresh Vector3D — this matters for code reuse later.

__repr__ must return a string.

Use return f"..." — not print. Returning None will give you ugly output again.

File layout.

Save as vector3d.py. One class per file is good practice — it makes imports clean.

When it's clean, send the file back here and I'll review.